

Kahoot: 15 questions

6 easy, 7 medium, 2 hard. The 2 hard ones are set to **double points**. Correct answer is marked; the note under each question is what to say after the reveal. Slide numbers refer to the talk deck.

EASY · 20 s · STANDARD POINTS · slide 6

1. For each request, what is a load balancer's core job?

A	Pick one healthy backend from a pool	correct
B	Cache the response	
C	Compress the response	
D	Encrypt the database	

Why: Health checks decide who is allowed; the algorithm picks which one. TLS, compression and WAF are features of the box, not of load balancing.

EASY · 20 s · STANDARD POINTS · slide 10

2. “Scale up” (vertical) means...

A	Move from 4 vCPU to 64 vCPU	correct
B	Run 16 pods of 4 vCPU each	
C	Add a second region	
D	Put a CDN in front	

Why: Scaling up needs no code change but has a ceiling and one power cord. Scaling out survives a dead pod, but needs a balancer in front and an app that keeps no state in memory.

EASY · 20 s · STANDARD POINTS · slide 6

3. Health checks decide who is allowed. What does the algorithm decide?

A	Which of the allowed backends gets this request	correct
B	Whether the pod is alive	
C	How long the DNS TTL is	
D	Which region the user hits	

Why: Two separate mechanisms. A pod failing its health check is out of the pool entirely, so the algorithm only ever chooses among the survivors.

EASY · 20 s · STANDARD POINTS · slides 13-14

4. An L4 balancer makes its decision using...

A	IP address and TCP/UDP port	correct
B	The HTTP path	
C	Cookies	
D	The request body	

Why: It chooses once, at the SYN, and writes (src ip, src port, dst ip, dst port) -> pod into a flow table. Packets 2..N carry no routing hint, so that table is the only memory of the choice.

EASY · 20 s · STANDARD POINTS · slide 35

5. Round robin picks a backend by...

A	Whose turn it is in a fixed rotation	correct
B	Which server has the fewest active connections	
C	A weight you set per server	
D	A hash of the client's IP	

Why: All four are real load balancing criteria, which is the trap: B is least connections, C is weighted round robin, D is IP hash. Round robin is one counter, $servers[i++ \% N]$, and knows only the order requests arrive - so a 3 s export and a 5 ms health check are both "one turn".

EASY · 20 s · STANDARD POINTS · slide 7

6. To the backend, a load balancer looks like...

A	The client	correct
B	The server	
C	A DNS resolver	
D	A router	

Why: To the client the balancer is the server; to the backend it is the client. The backend sees the balancer's IP, so the real one has to be passed in X-Forwarded-For (L7) or PROXY protocol (L4).

MEDIUM · 30 s · STANDARD POINTS · slide 17

7. Why should /healthz never query the database?

A	One 2 s DB blip fails every pod at once	correct
B	It is slow to write	
C	It leaks credentials	
D	The balancer ignores the body anyway	

Why: A shared dependency inside the check turns "degraded" into "100% down": every pod fails at the same moment and the pool empties. Keep the check shallow.

MEDIUM · 30 s · STANDARD POINTS · slide 37

8. With least connections, what happens to a brand-new server?

A	It has 0 connections and gets flooded	correct
B	It is never chosen	
C	It gets exactly 1/N	
D	It must be given a weight first	

Why: Zero active connections looks like “completely idle”, so it wins every comparison until it catches up. The fix is slow start: ramp it in instead.

MEDIUM · 30 s · STANDARD POINTS · slides 39-40

9. Your app keeps the shopping cart in pod memory. Every deploy logs users out. Best fix?

A	Switch to IP hash so each user sticks to one pod	
B	Turn on sticky session cookies on the load balancer	
C	Move sessions to Redis so any pod can serve any user	correct
D	Use least connections instead of round robin	

Why: Affinity only hides the problem: the pod still holds the cart, so replacing it during a deploy still loses it. IP hash also re-maps ~75% of users when the pod count changes (the rehashing problem, ch. 5). Alex Xu's fix in ch. 1 is shared session storage and a stateless web tier. Keep affinity afterwards only as a cache hint: losing it should cost a cache miss, not a logout.

MEDIUM · 30 s · STANDARD POINTS · slide 27

10. Anycast fails a site over by...

A	Withdrawing the BGP announcement	correct
B	Lowering the DNS TTL	
C	Returning HTTP 302	
D	Changing the IP address	

Why: Many sites announce the same prefix. Withdraw one and the internet re-learns the next-best path in seconds. No client needs a new answer, because the address never changed.

MEDIUM · 30 s · STANDARD POINTS · slide 25

11. What does a DNS-based GSLB never see?

A	How loaded a region is	correct
B	Whether a region is up	
C	Roughly where the resolver is	
D	Your routing policy	

Why: Its only inputs are the resolver IP and EDNS Client Subnet, its own health probes, a precomputed latency table, and your policy. A health check answers “is it up”, not “can it take more”, so a saturated healthy region keeps its full share.

MEDIUM · 30 s · STANDARD POINTS · slides 14-16

12. Which can an L7 balancer do that an L4 balancer cannot?

A	Re-send a failed request to another pod	correct
B	Route traffic by domain name	
C	Terminate TLS	
D	Eject a failing backend from the pool	

Why: Every option is a real balancer feature and L4 does three of them: it routes per domain by reading SNI in the clear, an NLB with a TLS listener terminates TLS, and health checks eject backends at both layers. Only the retry needs the request held in memory - L7 keeps a copy and can write it to a second pod, while L4 forwards each packet and forgets it.

MEDIUM · 30 s · STANDARD POINTS · slide 28

13. Why does a CDN help an API even at a 0% cache hit rate?

A	TCP and TLS handshakes finish ~30 ms away, not ~230 ms	correct
B	It compresses the JSON	
C	It caches the errors	
D	It lowers origin CPU	

Why: The edge PoP completes the connection itself, then forwards over a warm pooled connection on a private backbone. That handshake win lands even on requests that can never be cached.

HARD · 60 s · DOUBLE POINTS · slide 16

14. A large upload stops being retryable on an L7 balancer past the buffer cap. Why?

A	Past the cap it keeps no copy, so there is nothing to re-send	correct
B	The client cancels it	
C	The health check fails	
D	Re-encryption to the pod drops it	

Why: Retries exist only because the request sits in the proxy's memory. 10,000 concurrent requests x 64 KB is already 640 MB, so every balancer caps what it will buffer. Over the cap it streams the bytes straight through and retries quietly stop working.

HARD · 60 s · DOUBLE POINTS · slide 32

15. Peak 200k req/s global. A node does 25k. Survive one region down plus 1 of 3 AZs. Nodes/region?

A	12	correct
B	8	
C	16	
D	6	

Why: Failover moves traffic, not capacity, so size each region for the whole world: $200,000 / 25,000 = 8$. Then lose 1 AZ of 3, $\times 1.5 = 12$. They normally sit at ~33% of rated load, and that idle headroom is what makes failover a non-event.